



WEEKLY INTERNSHIP PROGRESS REPORT

Module: Week 4 – Advanced Laravel (REST APIs & Sanctum)

Submitted By: Muneeb ur Rehman

Role: Full-Stack Web Developer Intern

Stack/Focus: PHP, Laravel, MySQL, Bootstrap 5

Report Date: 22-Aug-2026

Submitted To: POF IT Dept

1. Executive Summary

The primary objective of Week 4 was to decouple the frontend from the backend by engineering a headless RESTful API. The week focused on API routing, stateless authentication using Laravel Sanctum, strict request validation, and data formatting via API Resources. The final deliverable is a fully documented, secure backend engine capable of serving structured JSON data to any modern frontend framework.

2. Key Achievements & Task Breakdown

- **Day 16: REST API Fundamentals** Transitioned from Monolithic MVC to a headless architecture. Initialized API routing in Laravel 11 (`php artisan install:api`) and established baseline JSON response endpoints to verify server uptime.
- **Day 17: API CRUD Operations** Engineered the `StudentController` as a dedicated API controller. Implemented robust validation for `POST` and `PUT` requests and returned standard HTTP status codes (`201 Created`, `200 OK`, `404 Not Found`, `422 Unprocessable Content`).
- **Day 18: API Authentication (Sanctum)** Secured the API endpoints by implementing stateless token-based authentication. Configured the `User` model with the `HasApiTokens` trait, developed custom registration/login endpoints, and locked down the `students` endpoints using the `auth:sanctum` middleware.
- **Day 19: API Resources** Implemented Laravel API Resources (`JsonResource`) to create a formatting layer between the database and the client. Successfully mapped database columns to frontend-friendly keys (e.g., mapping `name` to `full_name`) and secured the API by hiding internal timestamps and IDs where appropriate.
- **Day 20: Documentation & Testing** Tested all protected and public endpoints using browser-based HTTP `fetch` requests with `Authorization: Bearer` headers. Drafted comprehensive API documentation detailing endpoints, required payloads, and expected HTTP responses.

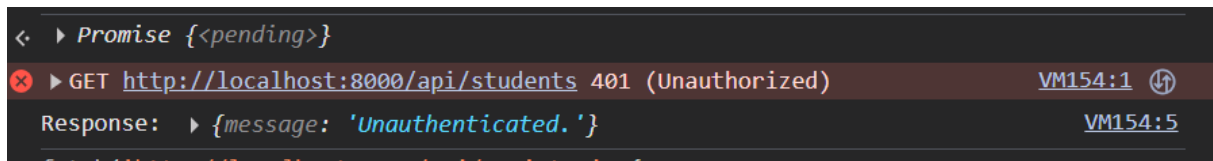
3. Technical Implementation Highlights

- **Stateless Authentication:** Successfully moved away from session-based cookies, utilizing Sanctum-generated plain text tokens passed via headers. This ensures the backend remains highly scalable and agnostic to the consuming client.
- **Data Encapsulation:** Utilized API Resources to strictly control the JSON schema exposed to the public, mitigating mass-assignment vulnerabilities and preventing data leaks of sensitive timestamps.
- **Environment Isolation:** Maintained a clean development environment by initializing a fresh `student_api` database and purging cached configuration files to prevent persistence bugs during architectural shifts.

4. Screenshot Documentation

The following screenshots demonstrate the successful execution and testing of the REST API endpoints.

Screenshot 1: Unauthenticated Request Rejection



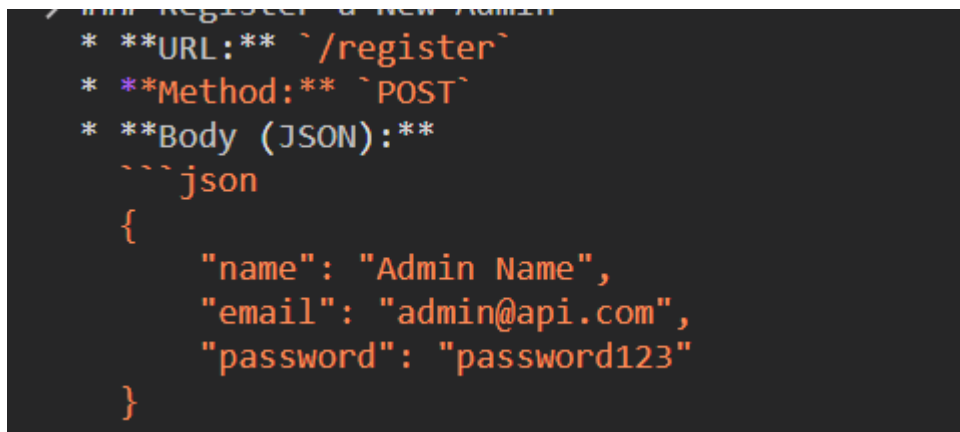
Description: Demonstrates the API returning a 401 Unauthenticated error when attempting to access the `/api/students` endpoint without a valid Bearer token.

Screenshot 2: Successful Authentication & Token Generation



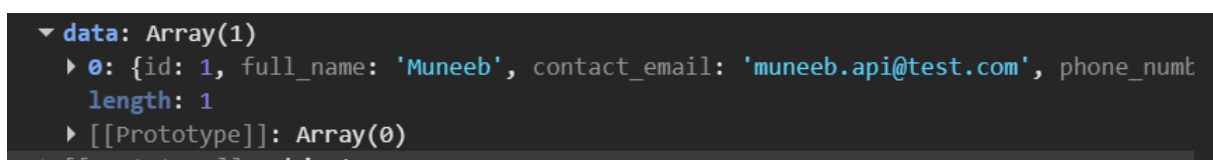
Description: Shows a successful POST request to the `/api/login` endpoint, returning a 200 OK status and the generated Laravel Sanctum plain text token.

Screenshot 3: Authorized POST Request (Creating a Resource)



Description: Displays a POST request to `/api/students` with the Bearer token attached in the headers. The response shows a 201 Created status with the newly inserted database record.

Screenshot 4: Formatted API Resource Output



Description: Shows a `GET` request to `/api/students`. The JSON output highlights the API Resource layer at work, displaying formatted keys (e.g., `full_name`, `contact_email`) and the intentional omission of backend timestamps.

5. Conclusion

Week 4 successfully demonstrated advanced proficiency in modern backend architecture. By constructing a secure, documented, and strictly typed RESTful API, the foundation is now set for integrating complex frontends or mobile applications. This concludes the core backend modules of the internship with a production-ready API service.